

SENTINELA

Sistema de Vigilância Orbital Inteligente para Defesa e Soberania

Prova de Conceito (POC) — Global Solution 2026.1

Tema: A Nova Economia Espacial e seu impacto positivo na Terra

FIAP — Inteligência Artificial (1TIAO) · Fase 7

Grupo 37

Integrante

Silvio Prestes Guerreiro Junior — RM 567958

Repositório:

github.com/silvioguerreiro/silvioguerreiro-1TIAOS_2026_FASE_7_Global_Solution_2

São Paulo · Junho de 2026

Sumário

1. Introdução

1.1 Contexto: a nova economia espacial

A exploração espacial deixou de ser apenas científica e tornou-se uma das maiores oportunidades tecnológicas, econômicas e estratégicas da atualidade. Satélites monitoram o clima, apoiam o agronegócio, realizam rastreamento global, auxiliam na prevenção de desastres e produzem enormes volumes de dados utilizados por governos, empresas e centros de pesquisa. Tecnologias originalmente desenvolvidas para missões espaciais impulsionaram avanços em Inteligência Artificial, sensores, automação, computação em nuvem, visão computacional e telecomunicações.

É nesse cenário que se insere a Global Solution 2026.1, que desafia os alunos a desenvolver soluções tecnológicas conectadas ao universo da economia espacial e capazes de gerar impacto positivo na Terra.

1.2 Problema e pergunta norteadora

O Brasil possui dimensão continental: cerca de 8,5 milhões de km² de território, aproximadamente 16,8 mil km de fronteiras terrestres e cerca de 5,7 milhões de km² de Águas Jurisdicionais Brasileiras (a chamada “Amazônia Azul”), além de abrigar a maior floresta tropical do planeta. Fiscalizar manualmente essa extensão é inviável. Os satélites geram um volume massivo de imagens, mas o verdadeiro gargalo está em transformar pixels em decisão acionável e em tempo hábil.

Pergunta norteadora: *como a Inteligência Artificial e as tecnologias digitais podem transformar a nova economia espacial e gerar impacto positivo na Terra?*

1.3 Objetivo e proposta de valor

O projeto SENTINELA é uma plataforma de vigilância orbital inteligente que utiliza IA para converter imagens de satélite e dados públicos em detecções geolocalizadas e alertas acionáveis. A POC concentra-se no monitoramento da Amazônia — detecção de mineração ilegal, queimadas e desmatamento — e foi projetada para escalar a outras aplicações estratégicas previstas no conceito: combate ao tráfico, atualização cartográfica, navegabilidade de rios, vigilância da Zona Econômica Exclusiva (ZEE) e controle de fronteiras, fortalecendo a defesa do espaço aéreo, das fronteiras terrestres e das Águas Jurisdicionais Brasileiras.

Impacto na Terra: proteção ambiental (Amazônia e clima), combate a crimes ambientais, resposta rápida a desastres naturais e reforço da soberania nacional — tudo a partir de dados públicos e com IA responsável e rastreável.

2. Desenvolvimento

2.1 Visão geral da solução

O SENTINELA é uma plataforma modular que encadeia ingestão de dados, núcleo de Inteligência Artificial, orquestração em nuvem, API e dashboard de comando, complementada

por sensores de campo (IoT) e uma camada de governança. O fluxo vai da imagem orbital até o alerta priorizado e a rota de fiscalização otimizada, integrando de forma coerente cerca de onze disciplinas do curso.

2.2 Arquitetura da solução

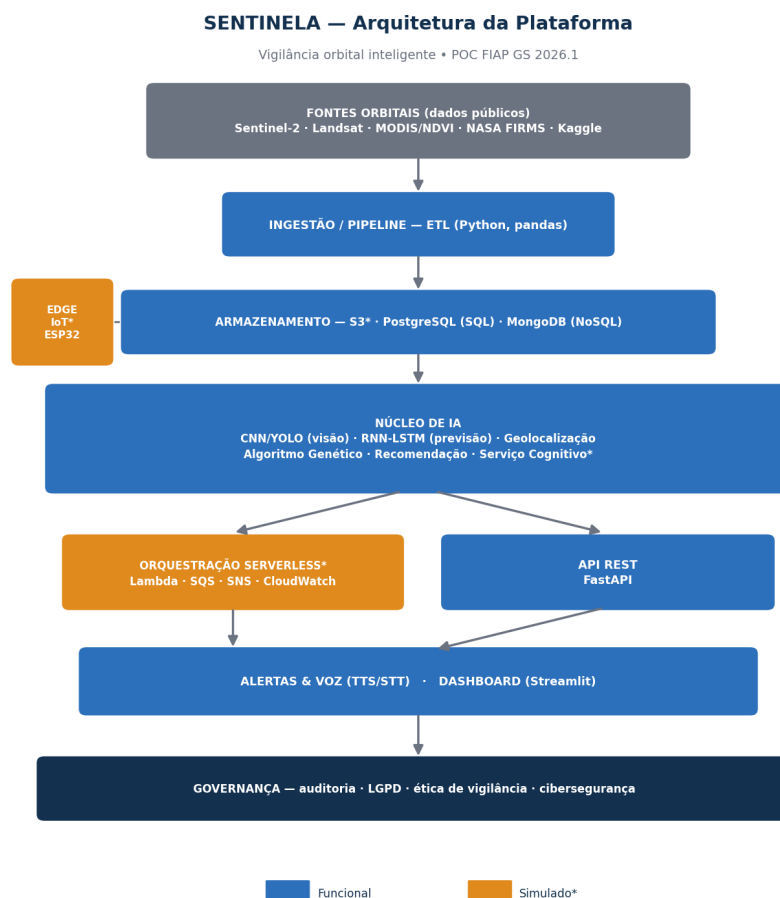


Figura 1 — Arquitetura em camadas do SENTINELA (azul = funcional; laranja* = simulado na POC).

Componentes principais:

- Fontes orbitais: dados públicos (Sentinel-2/Copernicus, Landsat, MODIS/NDVI, NASA FIRMS e datasets do Kaggle).
- Ingestão/Pipeline: ETL em Python que padroniza imagens e metadados.
- Armazenamento: objetos em S3 (simulado), detecções em PostgreSQL (SQL) e telemetria de sensores em MongoDB (NoSQL).
- Núcleo de IA: visão computacional (CNN/YOLO), previsão por RNN/LSTM, georreferenciamento, algoritmo genético, recomendação e serviço cognitivo (simulado).
- Orquestração serverless (simulada): funções Lambda disparadas por eventos, filas SQS, notificações SNS e logs de monitoramento (CloudWatch).
- API REST (FastAPI), Alertas por voz (TTS) e Dashboard de comando (Streamlit).

- Edge/IoT (simulado): sensor ESP32 de validação em solo (Wokwi).
- Governança: auditoria, conformidade com a LGPD, ética de vigilância e cibersegurança.

2.3 Fluxo de dados de ponta a ponta

A imagem de satélite chega à ingestão e é gravada no armazenamento, disparando uma função serverless simulada. O núcleo de IA executa a CNN para classificar cada célula da cena e detectar anomalias (mineração, queimada). Cada detecção é georreferenciada (convertida em latitude/longitude) e persistida em banco; a telemetria do sensor de campo valida o evento. A RNN/LSTM projeta a tendência dos focos, o recomendador prioriza os alertas por severidade e o algoritmo genético calcula a rota ótima de patrulha. A API expõe os resultados e o dashboard apresenta mapa, séries temporais e a fila de alertas, com aviso por voz ao operador. Toda ação é registrada na camada de governança.

2.4 Stack tecnológico e fontes de dados

Camada	Tecnologia	Disciplina (Fase·Cap)
Visão computacional	TensorFlow/Keras (CNN), OpenCV, YOLO	F6·C9-10
Série temporal	Keras LSTM/RNN	F7·C2, F4·C5
Otimização	Algoritmo Genético (Python)	F7·C4
Recomendação	scikit-learn / heurística	F6·C13
API	FastAPI + Uvicorn	F5/F6
Dashboard	Streamlit + mapas	F4·C4
Bancos	SQLite/PostgreSQL + MongoDB	F3·C7, F5·C3-5
Voz	pyttsx3 (TTS)	F7·C3
Nuvem (simulada)	moto/LocalStack (S3, Lambda, SQS, SNS)	F6·C12, F7·C5-6
Cognitivo (simulado)	Serviço estilo Rekognition	F7·C7
IoT (simulado)	ESP32 + Wi-Fi (Wokwi, C++)	F6·C3, F7·C8

Fontes públicas e citáveis: Copernicus/Sentinel-2, USGS Landsat, NASA FIRMS, MODIS NDVI, Kaggle (Airbus Ship Detection), recortes de uso do solo da Amazônia e EuroSAT.

2.5 Mapa de integração interdisciplinar

A integração entre disciplinas é o ponto central da nota. A tabela a seguir mostra onde cada eixo do curso entra na solução.

Disciplina / Eixo	Onde entra no SENTINELA
Visão Computacional / CNN / YOLO	Detecção de mineração, queimada e mudança de uso do solo
RNN / Séries Temporais	Previsão da evolução de focos de calor e desmatamento
Algoritmos Genéticos	Otimização da rota de patrulha e fiscalização
Ciência de Dados / Geolocalização	Pipeline, coordenadas e polígonos das detecções
SQL & NoSQL	PostgreSQL (detecções) + MongoDB (telemetria)
Cloud / Serverless / Monitoramento	Orquestração por eventos, filas e alertas
Serviços Cognitivos & Voz	Rotulagem auxiliar e alerta falado ao operador

Disciplina / Eixo	Onde entra no SENTINELA
IoT / ESP32	Sensor de validação em solo (Wokwi)
Dashboards	Painel de comando com mapa e séries
Cibersegurança & Direito Digital	Auditoria, integridade, LGPD e ética de vigilância

2.6 Módulos e principais trechos de código

Todo o código está em formato texto (sem prints), modularizado e pronto para rodar. A seguir, os trechos mais relevantes. O código completo está no repositório GitHub.

a) Visão computacional — classificador unificado (CNN Keras com fallback numpy):

```
def build_cnn(num_classes, img_size):
    from tensorflow.keras import layers, models
    m = models.Sequential([
        layers.Input((img_size, img_size, 3)),
        layers.Conv2D(16, 3, activation="relu", padding="same"),
        layers.MaxPooling2D(),
        layers.Conv2D(32, 3, activation="relu", padding="same"),
        layers.MaxPooling2D(),
        layers.Flatten(),
        layers.Dense(64, activation="relu"), layers.Dropout(0.3),
        layers.Dense(num_classes, activation="softmax"),
    ])
    m.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy", metrics=["accuracy"])
    return m

class VisionModel:
    # usa Keras se houver TF; senão fallback numpy
    def __init__(self, force_light=False):
        self.light = force_light or not HAS_TF
        self.model = _CentroidModel() if self.light else build_cnn(...)
```

b) Previsão de focos com RNN/LSTM (previsão recursiva de horizonte):

```
self.model = models.Sequential([
    layers.Input((self.look_back, 1)),
    layers.LSTM(32),
    layers.Dense(1),
])
self.model.compile(optimizer="adam", loss="mse")
# forecast recursivo dos próximos 'horizon' dias
for _ in range(horizon):
    yhat = self.model.predict(x[None, :, None])[0, 0]
    window.append(yhat); out.append(yhat)
```

c) Algoritmo Genético — rota ótima de patrulha (núcleo do laço evolutivo):

```
for _ in range(geracoes):
    scores = sorted((comprimento_rota(ind, pontos), ind) for ind in pop)
    nova = [s[1] for s in scores[:elite]] # elitismo
    while len(nova) < tam_pop:
        t1 = min(rng.sample(scores, 3), key=lambda t: t[0])[1] # torneio
        t2 = min(rng.sample(scores, 3), key=lambda t: t[0])[1]
        filho = _mutacao_swap(_ox(t1, t2, rng), taxa_mut, rng) # OX + mutação
        nova.append(filho)
    populacao = nova
```

d) Priorização de alertas (recomendação) e orquestração serverless por eventos:

```
def score_alerta(classe, conf, area_ha, em_area_protegida, fator_tend=0.0):
    sev = CLASS_SEVERITY.get(classe, 0.1)
    base = 0.45*sev + 0.20*conf + 0.20*min(area_ha/500, 1.0)
    return min(base + (0.15 if em_area_protegida else 0) + 0.15*fator_tend, 1.0)

bus.publish("nova_deteccao", reg) # dispara handlers (Lambda simulada)
```

2.7 O que é funcional (POC) e o que é simulado

A POC prioriza implementação real no núcleo de IA. Os componentes de infraestrutura que exigiriam conta AWS ou hardware físico são simulados com a mesma interface, acompanhados de plano de evolução.

Componente	Status	Disciplina
CNN de uso do solo (Keras + fallback numpy)	Funcional	F6·C9
Deteção na cena (janela deslizante)	Funcional	F6·C10
RNN/LSTM de previsão (+ fallback AR)	Funcional	F7·C2, F4·C5
Algoritmo Genético (rota)	Funcional	F7·C4
Recomendação / priorização	Funcional	F6·C13
Georreferenciamento (Haversine)	Funcional	F3·C6
API FastAPI + Dashboard Streamlit	Funcional	F4·C4
SQLite + MongoDB (mongomock)	Funcional	F3·C7, F5·C3-5
Alerta por voz (TTS)	Funcional	F7·C3
AWS Lambda/SQS/SNS/CloudWatch	Simulado	F6·C12, F7·C5-6
Serviço cognitivo (Rekognition)	Simulado	F7·C7
Sensor ESP32 de campo (Wokwi)	Simulado	F6·C3, F7·C8

Plano de evolução: (1) substituir os dados sintéticos por imagens reais Sentinel-2/Landsat e focos do NASA FIRMS; (2) treinar a CNN com transfer learning (MobileNetV2) em rótulos PRODES/DETER; (3) migrar a orquestração simulada para AWS real (S3, Lambda, SQS, SNS, Rekognition); (4) implantar o ESP32 físico com sensores de fumaça e GPS para validação em campo.

2.8 Resultados da execução do MVP

A plataforma foi executada de ponta a ponta e validada por uma bateria de testes automatizados (29 testes, todos aprovados). As figuras a seguir são saídas reais do sistema.

SENTINELA — Detecção de anomalias na cena orbital

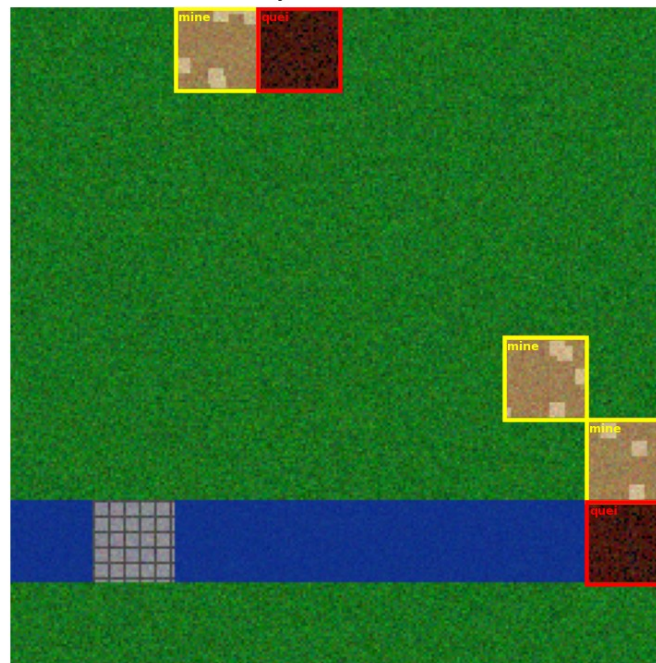


Figura 2 — Detecção de anomalias na cena orbital: mineração (amarelo) e queimada (vermelho).

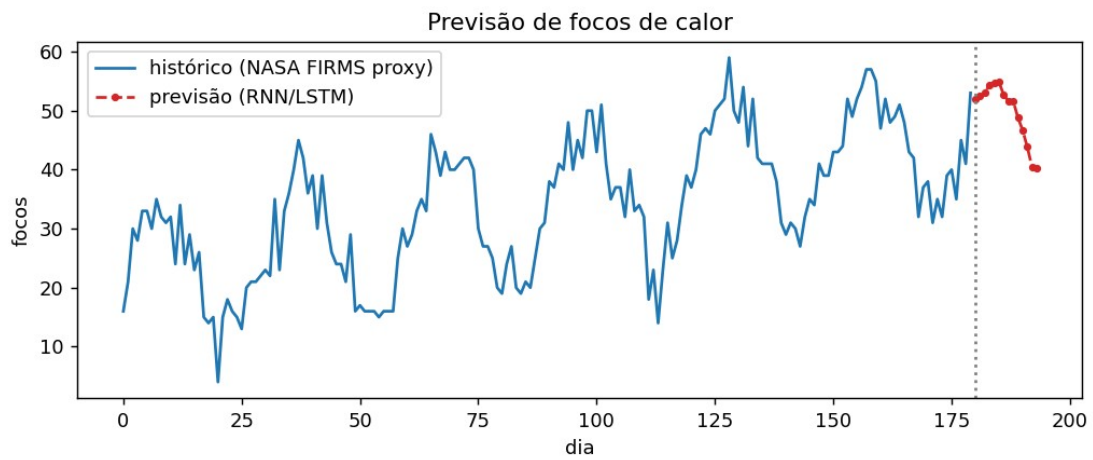


Figura 3 — Previsão de focos de calor (RNN/LSTM) para os próximos dias.

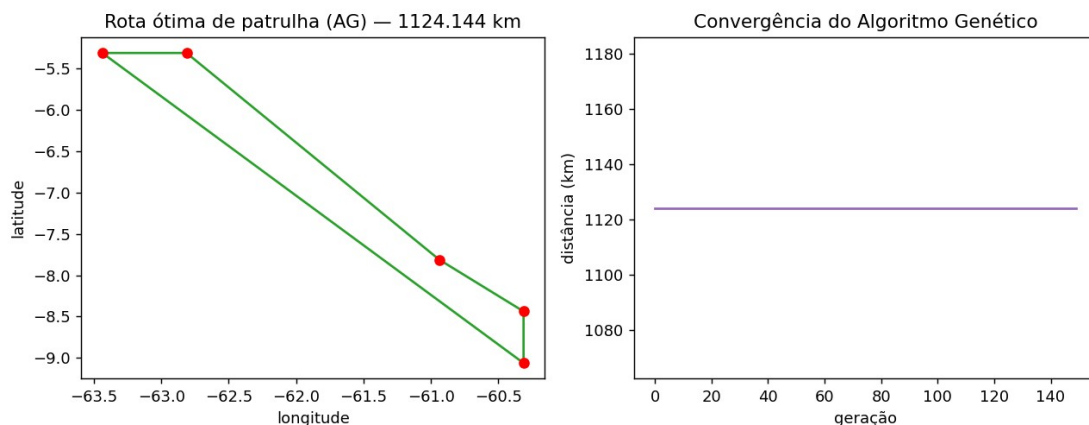


Figura 4 — Rota de patrulha otimizada por Algoritmo Genético e curva de convergência.

Métricas observadas: acurácia do classificador de 100% no conjunto de validação sintético; 5 anomalias detectadas e geolocalizadas; rota de patrulha 43% menor que a ordem ingênua (1.124 km contra 1.980 km); 17 invocações de função serverless e 5 notificações emitidas; alerta por voz gerado para o foco de maior prioridade.

Espaços para inserção de capturas reais do MVP em execução (recomendado para a entrega e o vídeo):



Figura — Dashboard de Comando (Streamlit): mapa operacional com AOI, área protegida, detecções por classe, rota de patrulha (GA) e focos FIRMS reais.

SENTINELA API 1.0 OAS 3.1

openapi.json

default

GET	/health	Health	⌵
GET	/deteccoes	Deteccoes	⌵
GET	/alertas	Alertas	⌵
GET	/previsao	Previsao	⌵
POST	/pipeline/run	Pipeline Run	⌵

Schemas

HTTPValidationError > Expand all object

ValidationError > Expand all object

Figura — Documentação interativa da API REST (FastAPI/Swagger em /docs).

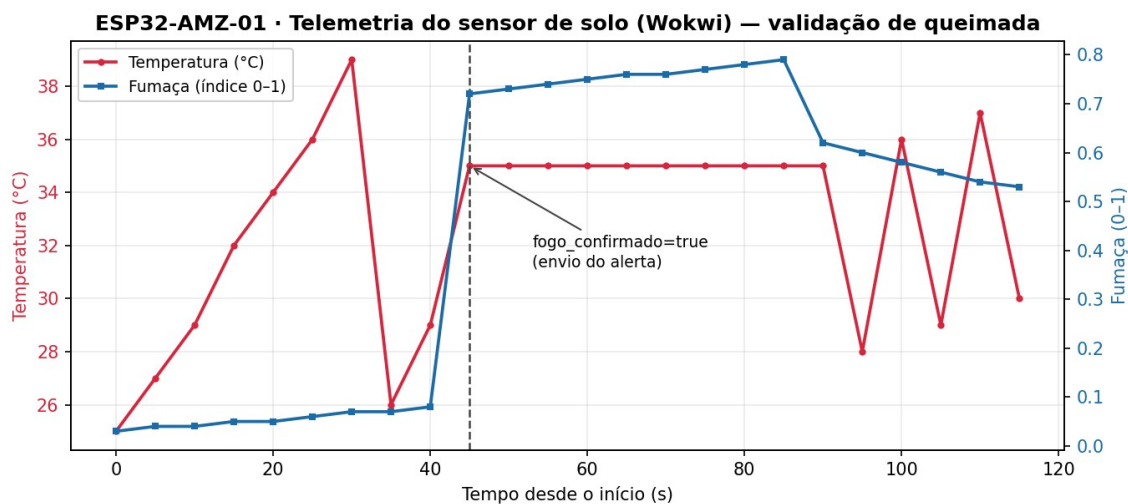


Figura 5 — ESP32 (Wokwi): telemetria real do sensor de solo (temperatura e fumaça) confirmando foco de queimada e disparando o alerta multi-fonte (sensor + detecção orbital).

2.9 Ética, governança e conformidade

Por se tratar de uma solução de vigilância, o SENTINELA adota princípios de IA responsável discutidos no curso (ética em IA e direito digital). O sistema opera sobre dados públicos e ambientais (uso do solo, focos de calor) e não realiza identificação de pessoas. Toda detecção é registrada com trilha de auditoria, e a plataforma observa a LGPD quanto a minimização e finalidade de dados. A camada de cibersegurança protege a integridade das detecções, evitando manipulação de evidências usadas pela fiscalização.

2.10 Dados públicos reais e repositório local (data lake)

A POC passou a consumir dados abertos reais, sem alterar o pipeline existente:

- NASA FIRMS (VIIRS 375 m): focos de calor por bounding box da AOI via Area API (chave gratuita). A função `fetch_fire_foci_firms()` normaliza a confiança (categórica l/n/h e numérica 0–100) e agrega em série diária para a previsão (LSTM/AR).

- Sentinel-2 L2A: cenas reais via STAC pública Earth-search (Element84/AWS), sem login. O filtro por cobertura (`s2:nodata_pixel_percentage`) retorna apenas tiles cheios; a imagem true-color pode ser tile-ada e consumida pela mesma CNN do projeto.

Para rodar offline-first e enriquecer a cada execução, os dados baixados são acumulados com deduplicação em um repositório local (data lake) em `src/data/lake/`, pelo módulo `data/repository.py`:

- Arquivos: `firms_foci.parquet` (focos), `s2_scenes.json` (catálogo de cenas) e `manifest.json` (contagens, período coberto e última sincronização).
- Funções `sync_*`() baixam em tempo real e agregam ao lake; `load_*`() leem o acumulado sem rede, alimentando previsão, mapas e visão computacional.

O dashboard de comando (Streamlit) passou a oferecer: seção Repositório local (sincronizar / usar offline), mapas interativos (pydeck) com AOI, área protegida, detecções por classe, rota de patrulha (algoritmo genético) e focos FIRMS sobrepostos, e um cartão Leitura para decisão em cada aba — que interpreta os achados (célula de maior concentração de focos, tendência da série temporal, confirmação multi-fonte ESP32 + órbita) e recomenda ações de fiscalização. Com isso, a solução evidencia engenharia de dados (ingestão por API, parsing, agregação e data lake) e análise em tempo real, mantendo o gerador sintético como fallback determinístico para reprodutibilidade.

3. Resultados Esperados

Espera-se que o SENTINELA reduza drasticamente o tempo entre a captura da imagem e a ação de fiscalização, priorizando automaticamente os focos mais críticos e otimizando o deslocamento das equipes. Em escala real, a plataforma pode apoiar órgãos ambientais e de defesa no monitoramento contínuo da Amazônia e das Águas Jurisdicionais Brasileiras, contribuindo para:

- Detecção precoce de mineração ilegal, queimadas e desmatamento, com geolocalização automática.
- Antecipação de tendências de focos de calor por meio de previsão temporal.
- Otimização logística das rotas de patrulha, reduzindo custo e tempo de resposta.
- Integração de sensores de campo para validação dos alertas e redução de falsos positivos.
- Base tecnológica reaproveitável para vigilância da ZEE, fronteiras, cartografia e navegabilidade.

4. Conclusões

O SENTINELA demonstra, como prova de conceito, que a Inteligência Artificial aplicada à economia espacial pode gerar impacto positivo e concreto na Terra. A solução integra de forma coerente visão computacional, redes neurais recorrentes, algoritmos genéticos, engenharia de dados, nuvem, serviços cognitivos, IoT, bancos SQL/NoSQL, dashboards e governança — articulando praticamente todo o conteúdo do curso em um caso de uso de alto valor estratégico e ambiental.

O núcleo de IA é funcional e testado; os componentes de infraestrutura que dependeriam de conta AWS e hardware foram simulados com a mesma interface e acompanhados de um plano claro de evolução para produção. Trata-se de uma base sólida, autoral e extensível, alinhada à pergunta norteadora da Global Solution 2026.1.

5. Repositório e reprodutibilidade

Repositório

GitHub:

https://github.com/silvioguerreiro/silvioguerreiro-1TIAOS_2026_FASE_7_Global_Solution_2

Execução (resumo):

```
pip install -r requirements.txt
python run_demo.py          # pipeline ponta a ponta (relatório no console)
python -m pytest             # bateria de testes (29 testes)
streamlit run dashboard/app.py # dashboard de comando
uvicorn core.api:app --reload # API REST (docs em /docs)
```

Observação: sem TensorFlow instalado, o sistema usa automaticamente os fallbacks em numpy, permitindo execução em qualquer máquina.

6. Referências (material do curso)

FIAP — Curso 1TIAO. Fases 1 a 7. Capítulos utilizados: F3·C6 (geolocalização); F4·C4 (Streamlit); F4·C5 (séries temporais e dados espaciais); F6·C3 (ESP32/visão); F6·C9 (CNN/Deep Learning); F6·C10 (detecção e segmentação); F6·C12 (serverless/Lambda); F6·C13 (sistemas de recomendação); F7·C2 (redes neurais recorrentes); F7·C3 (voz: fala e texto); F7·C4 (algoritmos genéticos); F7·C5-6 (AWS assíncrona, monitoramento e CloudFormation); F7·C7 (serviços cognitivos AWS); F7·C8 (POO no ESP32). Enunciado oficial: Tema Geral da Global Solution 2026.1.

VÍDEO DEMONSTRATIVO

Demonstração da solução em até 5 minutos (publicar no YouTube como “Não listado”).

Link do vídeo (YouTube — Não listado): <https://youtu.be/RHZ77HH8EM8>